```
data[:name]&.upcase&.strip&.split
```

The safe navigation operator is a great shortcut, but it's not a substitute for software design. If you have a lot of values and you don't know whether or not they are nil, it's worth thinking about whether there's a better way to structure the code.

Loops and Iterators

We've seen some discussion of Ruby iterators back in Chapter 4, Collections, Blocks, and Iterators, on page 53 when we talked about blocks and iterators. In this section, we'll talk about all of Ruby's looping constructs in more depth.

Loops

Ruby has primitive built-in looping constructs, separate from the iterator constructs we've already seen.

The most basic loop of all. Ruby provides a built-in iterator called loop:

```
loop do
  # block ...
end
```

The loop iterator calls the associated block forever (or at least until you break out of the loop, but you'll have to read ahead to find out how to do that).

The while loop executes its body zero or more times as long as its condition is true. For example, this idiom reads until the input is exhausted, assigning each line to the local variable line:

```
while (line = gets)
  # ...
end
```

The until loop is the opposite; it executes the body *until* the condition becomes true:

```
until play_list.duration > 60
  play_list.add(song_list.pop)
end
```

As with if and unless, you can use both of the loops as statement modifiers:

```
a = 1
a *= 2  while a < 100
a         # => 128
a -= 10 until a < 100
a         # => 98
```

Earlier, we said that a range can be used as a kind of flip-flop, returning true when some event happens and then staying true until a second event occurs. This facility is normally used within loops. In the example that follows, we read a text file containing the first ten ordinal numbers ("first," "second," and so on) but print only the lines starting with the one that matches "third" and ending with the one that matches "fifth":

```
file = File.open("ordinal")
while line = file.gets
```